Rooted: Bitcoin-Anchored Identity and Key Continuity

Michael Tidwell

Original idea and concept

October 19, 2026 | Version 0.1 | Research draft

Abstract

Public-key systems establish signing authority but do not inherently define identity continuity after key loss, replacement, or compromise. This paper presents Rooted, a proposed Bitcoin-anchored construction in which a stable genesis reference identifies a succession of application keys and Taproot control outputs.

A compact OP_RETURN record binds an application key to a control output. A confirmed spend of that output either advances the recognized state, explicitly revokes the identity, or terminates continuity when no valid continuation is provided. Taproot spending conditions authorize owner and guardian actions; client-side rules interpret those actions as identity transitions.

The construction uses existing Bitcoin primitives and proposes no consensus modification. It provides a candidate transaction encoding, a resolver model, and an analysis of recovery, compromise, privacy, and data-availability assumptions. Its claim is limited to verifiable authorized key succession: it does not establish personhood, prevent Sybil identities, guarantee recovery after control-key theft, or provide irreversible finality. The proposal remains a research draft; no production wallet, node-backed resolver, or independent security audit is presented.

Keywords: Bitcoin; Taproot; decentralized identity; key rotation; guardian recovery; revocation.

1. Introduction

Public-key accounts provide a persistent signing identity, but key replacement is not inherently equivalent to account continuity. A user may need to replace a device, recover lost signing material, or retire a compromised key while preserving application history and reputation.

A server can maintain an account database, but that server becomes an authority for the account's continuity. A bare public key removes that authority while leaving recovery and succession to each application.

The proposal separates three things:

- **Identity reference:** the permanent pointer to the identity's genesis output.
- **Application key:** the current key for messages, game actions, or authentication.
- **Control policy:** the Bitcoin spending conditions that authorize the next identity state.

The application key should normally be separate from Bitcoin control keys. Losing a frequently used game or Nostr key then need not give an attacker the ability to rotate the identity. A control policy can remain offline until a change is needed.

A domain name, profile, avatar, or payment address can be associated with this reference. None of those associations establishes a legal identity or a unique person.

2. System Model and Notation

Let I denote the stable identity reference, K_t the application public key at state t, U_t its current control outpoint, and q_t the state sequence. The active state is represented by $S_t = (I, U_t, K_t, q_t)$. The network component N of I is the network genesis-block hash. Rotation changes the state while preserving I.

The baseline has one identity per control UTXO: an unspent transaction output. Its stable reference is a tuple of **network genesis-block hash, genesis transaction ID, and control-output index**. Network binding prevents the same-looking reference from being interpreted on the wrong chain. Any shortened identifier notation used in the diagrams is illustrative; no textual identifier standard is assigned.

A creation transaction contains a zero-value OP_RETURN record and a funded Pay-to-Taproot control output. The record identifies that output and the initial application key. Change outputs and fee inputs are ordinary Bitcoin accounting, not identity state.

The Taproot output commits to an internal public key and a tree of spending conditions. A wallet can construct it with a verifiably derived NUMS internal point whose discrete logarithm is unknown, making the key path computationally unavailable under the usual cryptographic assumptions. It should retain evidence of that construction. The address alone cannot prove that no one knows a key-path secret. See BIP 341 [1].

The application key appears in the record; it need not be one of the spending keys. A Taproot address is an encoding of an output key, not a store of readable scripts. An unspent output hides its script tree. A script-path spend reveals the executed script, its inputs, and a control block, while unused branches stay hidden.

Bitcoin checks who may spend. Rooted checks what that spend means.

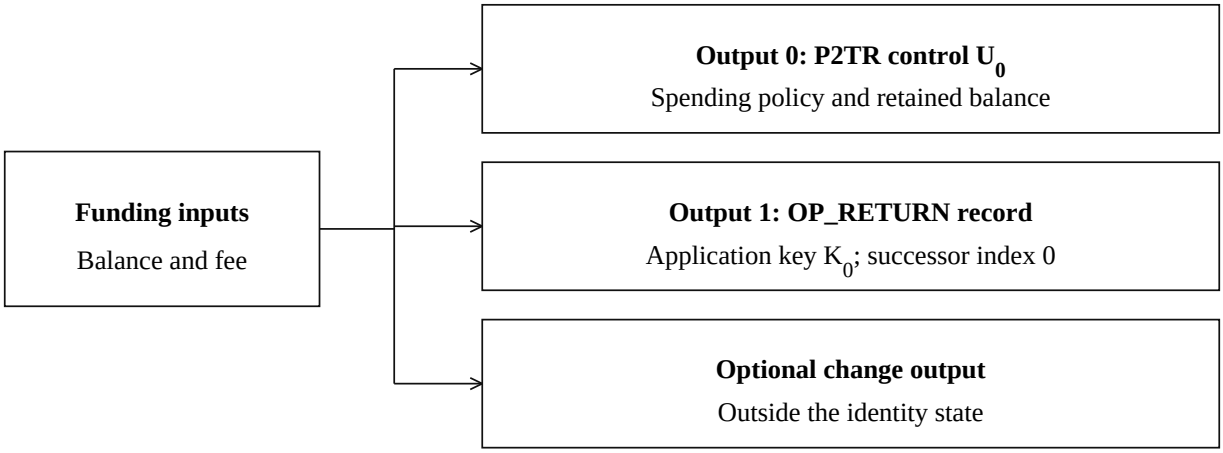

Figure 1. Creation transaction structure. The stable reference identifies the designated P2TR control output; the OP_RETURN record binds the application key to that output. Funding inputs and wallet change do not constitute identity state.

3. Identity State Transitions

Create. Confirm a transaction binding key K_0 to control output U_0. Its outpoint becomes the permanent reference. Creation alone claims no history before that transaction and proves no ownership of a real-world name.

Rotate. Spend the current control output. The same transaction carries a valid rotation record, the new application key K_1, and exactly one designated successor control output U_1. The resolver follows the spent outpoint to that successor. The identity reference remains unchanged.

Recover. Use an alternative authorized Taproot script, such as two of three guardians, to make the same rotation. Recovery is a spending-policy choice, not a separate on-chain operation. A guardian quorum has control power; it can also abuse that power.

Revoke. Spend the current output with an explicit revocation record and no successor. In this proposal, revocation is terminal on the selected chain. A new genesis creates a different identity; a link to it is an assertion, not restoration of the old one.

Unrecognized spend. If complete, validated transaction data shows that the current output was spent without a valid continuation or revocation, the v1 identity becomes terminated. Do not keep treating its old application key as current. If the data is unavailable, the resolver reports an unresolved state instead of inventing a transition.

A reorganization can undo any of these outcomes until the relevant transactions are sufficiently buried. "Terminal" refers to the history of the currently selected chain, not absolute finality.

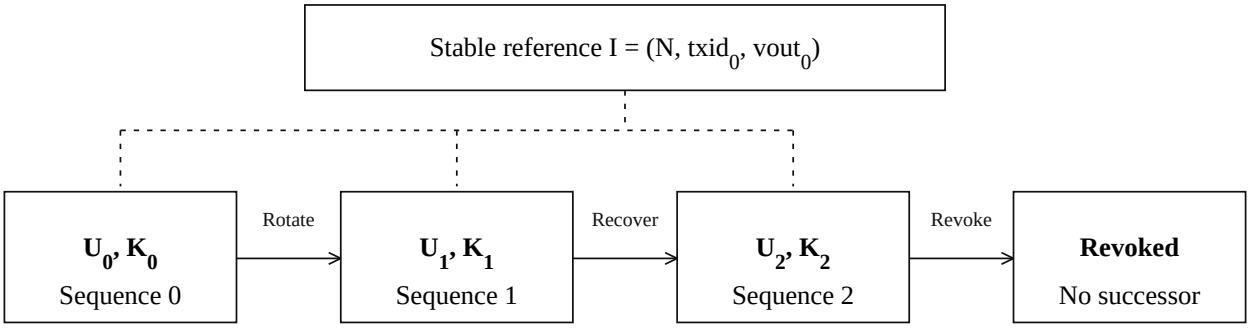

Figure 2. Illustrative identity-state succession. Rotation and guardian recovery replace the application key and control output while preserving the genesis reference. Dashed connections denote the unchanged reference. Explicit revocation creates no successor. Each transition is conditional on validation within the selected chain view.

4. Transaction Record Encoding

The proposed baseline uses a fixed-width version 1 transaction record. This is a candidate encoding, not an implemented network protocol. One transaction carries exactly one Rooted record and manages exactly one identity. Batching is deferred.

All integer fields below use unsigned little-endian encoding. The payload is exactly **78 bytes**. It is pushed as a single item in an `OP_RETURN OP_PUSHDATA1 0x4e <payload>` script, which is 81 bytes long. With an 8-byte zero value and a 1-byte script length, that output occupies 90 non-witness bytes.

Table 1. Candidate version 1 transaction-record encoding. Integer fields use unsigned little-endian encoding; the payload totals 78 bytes.

Offset	Field	Bytes	Meaning
0	Magic	4	ASCII `ROOT`; provisional namespace
4	Version	1	`0x01`
5	Operation	1	`0x00` create, `0x01` rotate, `0x02` revoke
6	Control input	2	Input index spending the current output; `0xffff` at creation
8	Successor output	2	Index of the new P2TR output; `0xffff` on revocation
10	State sequence	4	Zero at creation; previous sequence + 1 thereafter
14	Application key	32	Valid BIP 340 [2] x-only public key; all zeroes on revocation
46	Document hash	32	SHA-256 of exact associated-document bytes; all zeroes if absent

The document hash can bind a profile or a policy manifest. Availability is separate from integrity: Bitcoin does not host the document. Version 1 commits to exact bytes, avoiding an implied JSON canonicalization standard. The document must carry its own schema and context; application signatures must separately bind their protocol and network.

An active successor must have a native P2TR scriptPubKey (`OP_1` followed by a 32-byte output key that lifts to a secp256k1 point). The index must be in range. Sequence overflow ends the v1 lineage; it must never wrap to zero.

For creation, require operation 0, sequence 0, control input `0xffff`, a valid application key, and a valid successor. For rotation, require operation 1, the exact current outpoint at the referenced input, sequence + 1, a valid new key, and a valid successor. For revocation, require operation 2, that same current input, sequence + 1, successor `0xffff`, and zero key and document fields. Reject extra or duplicate Rooted records and noncanonical framing.

A complete transition also requires a Bitcoin-valid script-path spend of the control output. A key-path spend is an unrecognized spend under this v1 profile. Wallets should sign control inputs with `SIGHASH_DEFAULT` or `SIGHASH_ALL` so their signatures commit to the complete output vector, including the record and successor. Weak sighash modes need separate security analysis and are outside the proposed wallet profile.

These are **client rules**, not covenants. The scripts shown here do not force a spender to create a successor, preserve funds, or publish this record. A spend can be valid Bitcoin and still terminate a Rooted identity. Accordingly, the output does not enforce the complete identity state machine at the consensus layer.

The 78-byte payload is intentionally small enough for the historical 83-byte data-carrier script policy. That policy was never an 80-byte consensus cap: Bitcoin Core 30.0 changed its defaults to permit larger and multiple OP_RETURN outputs. Relay and mining settings vary. See the Core 30.0 release notes [3].

5. Taproot Authorization Policies

Taproot offers a compact commitment to alternative ways of authorizing a spend. An illustrative control policy might offer an owner branch and a two-of-three guardian branch. Bitcoin accepts a spend satisfying either branch. The identity client then interprets its record.

An illustrative guardian leaf is:

```
<G1> OP_CHECKSIG
<G2> OP_CHECKSIGADD
<G3> OP_CHECKSIGADD
OP_2 OP_NUMEQUAL
```

Listing 1. Illustrative two-of-three Tapscript authorization leaf.

Here G1-G3 are valid 32-byte x-only secp256k1 public keys. Witness signature positions are supplied in reverse key order, with empty elements for nonsigners. This is explanatory script, not a production descriptor or wallet implementation. Tapscript disables legacy CHECKMULTISIG and supplies CHECKSIGADD; see BIP 342 [4].

A timelocked guardian branch can add `OP_CHECKSEQUENCEVERIFY` or `OP_CHECKLOCKTIMEVERIFY`. Relative delays require the transaction-version and sequence conditions of BIP 68 [5] and BIP 112 [6]; absolute constraints follow BIP 65 [7]. These conditions make a spend eligible. They do not schedule a transaction or notify its owner.

A simple delayed guardian branch does not provide a veto window after a recovery request: its delay runs from the control output's confirmation. Once eligible, it can race an owner spend. A real challenge-and-cancel recovery flow would need additional transaction stages and a separately analyzed policy.

Large threshold policies are possible in principle, subject to script, stack, signature-budget, weight, and relay limits. "Arbitrary quorum" is not a blanket assurance. A two-of-three policy is a better first prototype than an untested 20-of-30 design.

Unused leaves remain private, but a revealed CHECKSIGADD branch exposes the public keys in that leaf and the signature slots that succeeded. A specialized threshold signature can hide individual participation behind one aggregate key, with additional setup and nonce-safety requirements. MuSig2 is an n-of-n construction, not a drop-in arbitrary threshold protocol; see BIP 327 [8]. Shamir secret sharing alone does not provide threshold signing: reconstruction brings a full secret together.

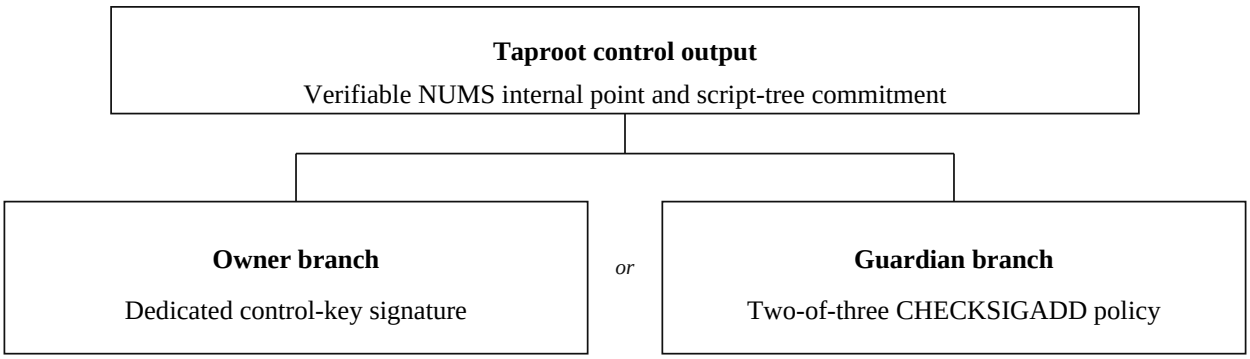

Figure 3. Alternative script-path authorization conditions for a control output. The NUMS construction must be verifiable. Either branch authorizes spending, but neither branch enforces publication of a valid identity record or inheritance of the recovery policy by a successor.

6. Verification and Chain-State Resolution

A resolver starts with the network-bound genesis outpoint, validates its creation record, and follows each confirmed spend of the current control output. At each step it checks the Bitcoin transaction, relevant script-path witness, canonical record, input binding, sequence, and successor. Ordinary inputs or change outputs do not advance the identity.

```
resolve(network, genesis, validated_chain_tip):
  state = validate_creation(genesis)
  while current_control_output_has_a_confirmed_spend(state):
    tx = obtain_complete_validated_spend(state.outpoint)
    if tx data is unavailable: return UNRESOLVED
    if recognized_revocation(tx, state): return REVOKED
    if not recognized_rotation(tx, state): return TERMINATED
    state = successor_state(tx, state)
  return ACTIVE(state, validated_chain_tip)
```

Algorithm 1. Resolution over a complete, validated chain view.

The validation assumption is essential. The recommended reference resolver uses a fully validating Bitcoin node plus an index of spends and Rooted records. A cached index is reconstructible, but still necessary infrastructure for efficient discovery. A pruned node may need external historical blocks and witnesses to rebuild it. Bitcoin Core's transaction index alone is not a general outpoint-spend index.

A light client can check inclusion proofs, but a conventional txid Merkle proof does not authenticate witness bytes. Witness authentication needs the SegWit commitment through the coinbase and the associated proof material, or a stronger trusted/full-validation source. BIP 141 [9] defines that commitment. Such proofs also do not establish that no later spend was omitted. Headers alone cannot establish the latest identity state.

A scan beginning at an advertised launch height discovers only identities created at or after that height. It is an explicit protocol boundary or a checkpoint assumption, not a way to reconstruct arbitrary older histories from headers.

Each resolver response should include the network, validated tip hash and height, identity genesis, current outpoint, current application key, sequence, observed state, confirmation count, and data-source assumptions. Clients should display pending changes separately and retain reversible state for reorgs. This draft suggests six confirmations as a configurable example for sensitive actions, not a finality guarantee or a universal security threshold.

7. Security and Trust Assumptions

Application-key theft. An attacker can forge new application messages until clients recognize a confirmed rotation. If control keys are separate, stealing the application key alone cannot spend the identity output. Rotation limits future acceptance; it cannot erase previously signed content or reliably date when a secret was stolen. An attacker can backdate an ordinary signed timestamp. Applications need authenticated historical key state and independent time evidence for historical claims.

Control-key theft. An attacker who satisfies a control branch can rotate to their own keys or terminate the lineage. If an independent recovery branch still applies to the current output, the owner may race them. After a malicious rotation confirms to an attacker-only successor, the original guardians have no inherent authority over that new output. Basic Taproot scripts do not carry a mandatory recovery branch forward.

Unproven key claims. Anyone can publish another person's application public key in a creation record. The anchor alone does not prove possession of that key or authorize taking over an existing account. Applications should authenticate onboarding, pin the intended genesis reference, and require a domain-separated challenge signature from the application key. Never choose a user's identity by taking the first on-chain record that mentions their key.

Guardian collusion. Recovery authority is custody of a power, even when the guardians do not hold a conventional wallet balance. Choose quorums and backups accordingly. A visible guardian key shows an authorization relationship, not an endorsement or a social trust score.

Bounty output. Funding the output with 50,000 sats creates something an attacker may take; it does not guarantee disclosure, detect application-key theft, or distinguish theft from an authorized spend. It can increase attack incentives. Keep bounties outside the baseline security argument.

Loss and outages. Lose all viable signing material or all necessary script/control-block backups and control may become inaccessible. An unavailable indexer should produce an unresolved result. It must not cause a client to silently accept an old key as current.

Privacy and Sybil resistance. Public succession makes keys linkable. Explicit guardian scripts may reveal relationships. Fees increase creation cost but do not establish one person per identity. Avoid placing personal information or reusable payment addresses in permanent metadata.

Backups. Retain the full policy descriptor or equivalent tree data, derivation information, all needed scripts and control blocks, relevant key material or signer access, and state-discovery instructions. BIP 386 [10] describes Taproot output descriptors. Encrypt sensitive backups and test recovery before funding. "Save the address" is not a recovery plan.

8. Applications and Related Work

A small application is the right proving ground. A game can attach inventory history and reputation to the stable reference, accept actions signed by the current application key, and support an explicit recovery process. It should resolve state at login and before high-value actions. Users can create and recover test identities without putting mainnet funds at risk.

For Nostr, this is a proposed optional succession layer. Existing clients use signed events and public keys under NIP-01 [11]. They will not automatically transfer followers or interpret a Bitcoin rotation. A client extension and explicit follow policy would be required. Do not equate "one key" with "one-time identity."

NIP-05 [12] maps an internet identifier to a Nostr key through a domain's well-known endpoint. A Bitcoin anchor could complement discovery, but is not itself NIP-05, a human-readable name, or proof of a domain. An endpoint outage does not invalidate Nostr signatures or erase the underlying key identity.

An AI agent could use the same succession reference across software upgrades, but continuity proves authorized control, not that the same model, operator, behavior, or intentions persist. Payment endpoints and other-chain identifiers should be typed, signed bindings with explicit scope and expiry. A mapping to Liquid does not import Bitcoin's security assumptions into Liquid or create cross-chain finality.

This sketch has substantial prior context. DID Core [13] separates identifiers, controllers, and verification methods. Sidetree [14] and did:btcr2 [15] explore ledger-anchored identity and updates. This proposal should be compared with that work before asserting a distinctive contribution. Taproot's arrival does not imply earlier DID systems lacked rotation or recovery.

9. Costs, Limitations, and Implementation Requirements

Every creation or update consumes blockspace. The live control output also occupies the UTXO set and holds spendable bitcoin. An OP_RETURN output is provably unspendable; it does not become a live UTXO. The control balance must satisfy the relevant dust and relay rules and a wallet needs a practical fee-funding strategy.

The record output alone occupies 90 vbytes. A realistic complete update also includes inputs, a Taproot witness, a successor, and possibly change. The cost illustration assumes a **250-vbyte transaction**; this is an analytical example rather than a measured benchmark or a live fee estimate. At 10 sat/vB that would cost 2,500 sats, excluding the retained control balance. Actual weight depends on the selected leaf, Merkle depth, signers, funding, and outputs.

Start with a regtest wallet and resolver for one identity per transaction, one owner branch, and one two-of-three guardian branch. Test the complete lifecycle, malformed records, lost backups, competing spends, fee replacement, missing history, and reorganizations. Then integrate a game login that refuses stale or unresolved key state. Mainnet deployment of the protocol is not proposed at this stage.

Before interoperability, freeze the namespace and wire format; specify the full accepted policy and sighash profile; publish Bitcoin transaction, witness, and resolver test vectors; independently review scripts; and measure real fees and recovery usability. Batching, cross-chain continuity, threshold privacy, and formal DID-method integration remain separate research tracks.

The accompanying repository includes a tested educational state-machine model and record codec. It does **not** contain a Bitcoin wallet, transaction signer, script interpreter, node-backed resolver, or an independent security audit. Its technical review is a source-based design review. An executable regtest prototype and independent protocol review are prerequisites for evaluating production suitability.

10. Conclusion

Rooted separates the stable identity reference, the application signing key, and the Bitcoin policy governing changes in control. This separation provides a model for application-level key continuity using confirmed Bitcoin spends and explicit resolver rules. Its security depends on those rules, complete and current chain data, sound Taproot policies, and recoverable signing material. The next research milestone is a regtest implementation with interoperable transaction vectors, measured costs, and independently reviewed wallet and resolver behavior.

The idea and concept are entirely Michael Tidwell's. "Rooted" remains a working title. This paper does not assert novelty, standards compliance, peer review, or institutional endorsement.

References

[1] Bitcoin Improvement Proposals. BIP 341: Taproot: SegWit version 1 spending rules. https://bips.dev/341/ (accessed October 19, 2026).

[2] Bitcoin Improvement Proposals. BIP 340: Schnorr Signatures for secp256k1. https://bips.dev/340/ (accessed October 19, 2026).

[3] Bitcoin Core. Bitcoin Core 30.0 release notes. https://bitcoincore.org/en/releases/30.0/ (accessed October 19, 2026).

[4] Bitcoin Improvement Proposals. BIP 342: Validation of Taproot Scripts. https://bips.dev/342/ (accessed October 19, 2026).

[5] Bitcoin Improvement Proposals. BIP 68: Relative lock-time using consensus-enforced sequence numbers. https://bips.dev/68/ (accessed October 19, 2026).

[6] Bitcoin Improvement Proposals. BIP 112: CHECKSEQUENCEVERIFY. https://bips.dev/112/ (accessed October 19, 2026).

[7] Bitcoin Improvement Proposals. BIP 65: OP_CHECKLOCKTIMEVERIFY. https://bips.dev/65/ (accessed October 19, 2026).

[8] Bitcoin Improvement Proposals. BIP 327: MuSig2 for BIP340-compatible Multi-Signatures. https://bips.dev/327/ (accessed October 19, 2026).

[9] Bitcoin Improvement Proposals. BIP 141: Segregated Witness (Consensus layer). https://bips.dev/141/ (accessed October 19, 2026).

[10] Bitcoin Improvement Proposals. BIP 386: tr() Output Script Descriptors. https://bips.dev/386/ (accessed October 19, 2026).

[11] Nostr Protocol. NIP-01: Basic protocol flow description. https://github.com/nostr-protocol/nips/blob/master/01.md (accessed October 19, 2026).

[12] Nostr Protocol. NIP-05: Mapping Nostr keys to DNS-based internet identifiers. https://github.com/nostr-protocol/nips/blob/master/05.md (accessed October 19, 2026).

[13] World Wide Web Consortium. Decentralized Identifiers (DIDs): Core specification. https://www.w3.org/TR/did/ (accessed October 19, 2026).

[14] Decentralized Identity Foundation. Sidetree Protocol specification. https://identity.foundation/sidetree/spec/ (accessed October 19, 2026).

[15] did:btcr2 project. Bitcoin-anchored decentralized identifier method specification. https://github.com/dcdpr/did-btcr2 (accessed October 19, 2026).

[16] Internet Research Task Force. RFC 9591: The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures. https://www.rfc-editor.org/rfc/rfc9591.html (accessed October 19, 2026).

Appendix A. Review of Technical Assumptions

Table A1 summarizes the source-based design review. "Supported" identifies a supported building block; "Qualified" denotes a claim requiring additional assumptions; "Corrected" identifies an interpretation that must not be treated as a property of the construction; and "Open" identifies unresolved implementation work. This analysis is not an independent security audit.

Table A1. Assessment of technical claims and limitations.

Claim examined	Assessment	Technical finding and primary references
Anchor a key to a Bitcoin output	Supported	A transaction can bind a key and an output index in OP_RETURN. The proposed identity meaning is enforced by clients. [1], [3]
Taproot enforces rotation and revocation	Corrected	Taproot validates spending conditions. The proposed record and resolver define the action and successor. A Bitcoin-valid spend can end continuity. [1], [4]
A NUMS key forces a script reveal	Qualified	A verifiably derived point with unknown discrete logarithm makes key-path use computationally unavailable. An address alone does not prove its construction. [1]
Timelocks enable automatic rotation	Corrected	Timelocks constrain eligibility. Someone must still construct, sign, fund, and broadcast a transaction. [6]
A delayed recovery branch provides a veto period	Corrected	A simple relative lock runs from output confirmation, not from a later recovery request. Challenge-and-cancel recovery needs additional stages. [6]
Guardian recovery is a web of trust	Corrected	Guardian keys have control authority. Social endorsement and reputation require separate signed assertions and application rules. [4], [13]
Recovery does not reveal which guardian signed	Qualified	A revealed CHECKSIGADD leaf exposes its keys and signing slots. Hiding participants requires a separately analyzed aggregate or threshold construction. [4], [8], [16]
Shamir shares are threshold signing	Corrected	Secret sharing alone reconstructs a secret. It does not supply distributed signature generation or its nonce-safety guarantees. [16]
An on-chain anchor is NIP-05	Corrected	NIP-05 is a domain-to-key mapping. An anchor can complement discovery; existing Nostr clients do not inherit succession support. [12], [11]
Headers are enough; no indexing is needed	Corrected	Resolvers need spends, records, and relevant witnesses. Inclusion does not prove latest state; witness authentication needs more than a txid Merkle branch. [9]
Any public/private key can control the output	Qualified	Taproot signature checks use BIP 340 secp256k1 keys. Version 1 application keys use the same format; other key types need an extension. [2], [4]
A bounty proves a key was compromised	Corrected	A spend proves the spending conditions were satisfied, not that theft occurred. An attacker may keep a stolen key secret or use a valid rotation. [1]
Recovery survives any malicious rotation	Corrected	The successor policy can omit the old guardians. The baseline scripts do not enforce policy inheritance across outputs. [1], [4]
80 bytes is an OP_RETURN consensus limit	Corrected	That number described historical relay policy. Core 30.0 changed its defaults; deployment still depends on actual relay and mining policies. [3]
Continuity proves the same person	Corrected	It proves authorized succession under a policy. Keys and control can be transferred; multiple identities can belong to one operator. [13]
The design is ready for mainnet	Open	A candidate record and educational model are provided. Regtest transactions, reviewed scripts, a real resolver, interoperability vectors, and independent review remain.